

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА» (СПбГУТ)
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ
ИНЖЕНЕРИИ (ИТПИ)
КАФЕДРА ПРОГРАММНОЙ ИНЖЕНЕРИИ И ВЫЧИСЛИТЕЛЬНОЙ
ТЕХНИКИ (ПИиВТ)

Дисциплина: «Разработка приложений искусственного интеллекта в
киберфизических системах»

Лабораторная работа №4.

Тема: «Аппроксимация функции при помощи искусственных нейронных
сетей»

Выполнил:
студент группы ИКПИ-32
Филимонов Д.Е.

Принял:
Березкин А. А.
Подпись _____

Санкт-Петербург
2026 г.

Цель работы

Изучение возможности аппроксимации функций с помощью нейронной сети прямого распространения (многослойного персептрона), реализация алгоритма обратного распространения ошибки, исследование качества аппроксимации на примере заданной функции.

Ход работы

Параметр	Значение
Функция	$\sin(x) / (1 + \exp(-x))$
Промежуток	$x \in [0, 10]$
Контрольный вопрос	4

Аппроксимация — это замена одной функции другой, более простой или удобной для исследования, которая проходит близко к исходным данным. В отличие от интерполяции, аппроксимация не требует точного прохождения через все точки, что особенно важно при наличии шума в данных.

Квадратичный критерий близости (наименьших квадратов) имеет вид:

$$R = \sum_{i=1}^n \beta_i (y_i - g_i)^2$$

где y_i — заданные значения, g_i — значения аппроксимирующей функции, β_i — весовые коэффициенты.

Многослойный персептрон — это нейронная сеть, состоящая из входного слоя, одного или нескольких скрытых слоёв и выходного слоя. Согласно теореме Фуннахаши (1989 г.), такая сеть с одним скрытым слоем может аппроксимировать любую непрерывную функцию с любой точностью при достаточном количестве нейронов в скрытом слое.

Алгоритм обратного распространения ошибки (Back Propagation) — основной метод обучения многослойных сетей. Он основан на градиентном спуске: веса корректируются в направлении, противоположном градиенту функции ошибки. Формула обновления весов:

$$w_{ij}(t+1) = w_{ij}(t) - \eta \frac{\partial E}{\partial w_{ij}}$$

где η — скорость обучения, E — функция ошибки (обычно среднеквадратичная). Для ускорения сходимости и предотвращения застревания в локальных минимумах часто используется метод момента (momentum), добавляющий к обновлению часть предыдущего приращения.

Для генерации обучающей выборки было взято 200 точек (с шагом 0,05) на интервале $x \in [0, 10]$. К каждой точке функции был добавлен случайный шум с амплитудой 0,03 для имитации реальных экспериментальных данных. После нормализации данные были разделены на обучающую (80%, 160 точек) и валидационную (20%, 40 точек) выборки.

Табличное представление

№	x	$f(x) = \sin(x)/(1+\exp(-x))$
1	0,00	0,0000
2	0,05	0,0499
3	0,10	0,0993
4	0,15	0,1480
5	0,20	0,1956
6	0,25	0,2417
7	0,30	0,2859
8	0,35	0,3277
9	0,40	0,3667
10	0,45	0,4025

Графическое представление

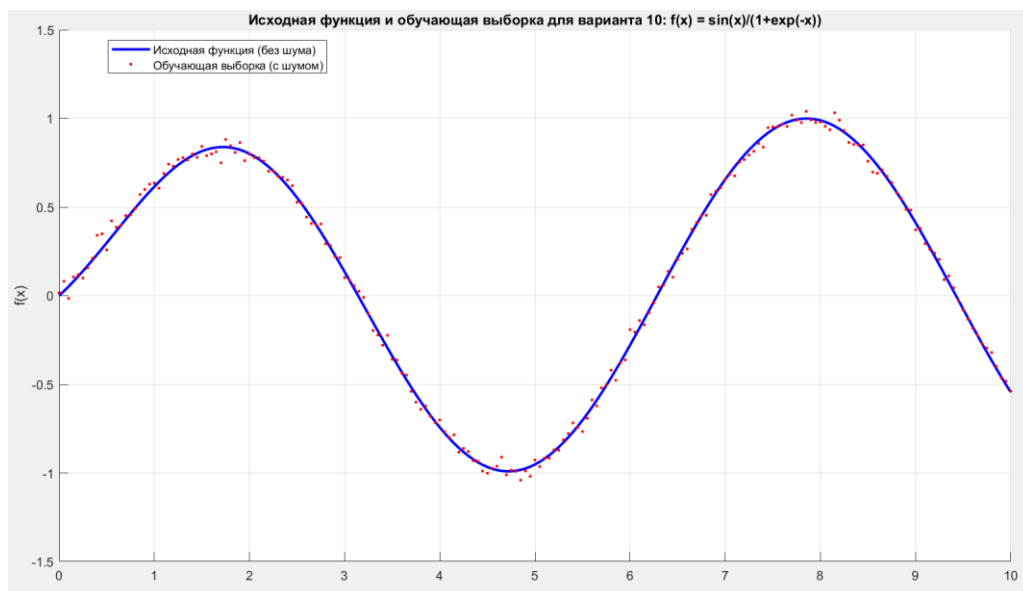


Рисунок 1 – Обучающая выборка

На рисунке 1 представлены график исходной функции (синяя сплошная линия) и точки обучающей выборки с добавленным шумом (красные точки). Видно, что шум создаёт некоторый разброс относительно истинной функции, но общая тенденция сохраняется.

Архитектура нейронной сети

В работе использовалась двухслойная нейронная сеть прямого распространения со следующей архитектурой:

- Входной слой: 1 нейрон (принимает значение аргумента x)
- Скрытый слой: 10 нейронов с сигмоидальной функцией активации
- Выходной слой: 1 нейрон с линейной функцией активации

Функции активации:

- Сигмоида (логистическая): $f(s) = \frac{1}{1+e^{-s}}$, её производная $f'(s) = f(s)(1 - f(s))$
- Линейная (purelin): $f(s) = s$, производная равна 1

Параметры обучения:

- Скорость обучения (learning rate): 0,1
- Момент (momentum): 0,9
- Максимальное количество эпох: 2000
- Целевая ошибка: 10^{-4}
- Ранняя остановка: при отсутствии улучшения ошибки валидации в течение 100 эпох

Процесс обучения

Обучение сети проводилось методом стохастического градиентного спуска с использованием алгоритма обратного распространения ошибки. На каждой эпохе примеры предъявлялись сети в случайном порядке, после каждого примера производилась коррекция весов.

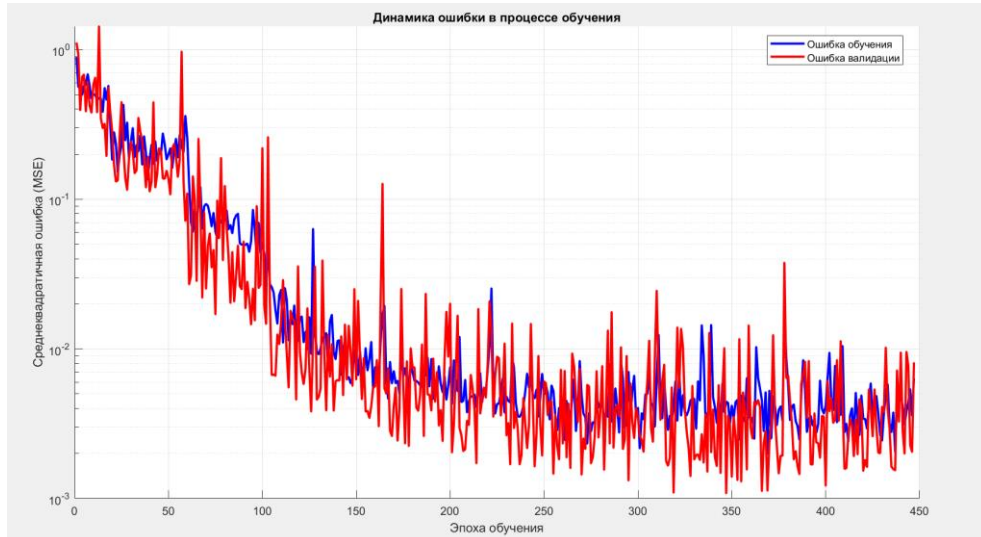


Рисунок 2 – Динамика ошибки

Как видно из графика на рисунке 2, ошибка обучения и ошибка валидации быстро уменьшались в первые 200 эпох, после чего процесс стабилизировался. На эпохе 447 была зафиксирована ранняя остановка, так как ошибка валидации перестала уменьшаться. Финальная среднеквадратичная ошибка (MSE) на обучающей выборке составила 0,006357, а наименьшая ошибка валидации достигла значения 0,001216.

Результаты аппроксимации

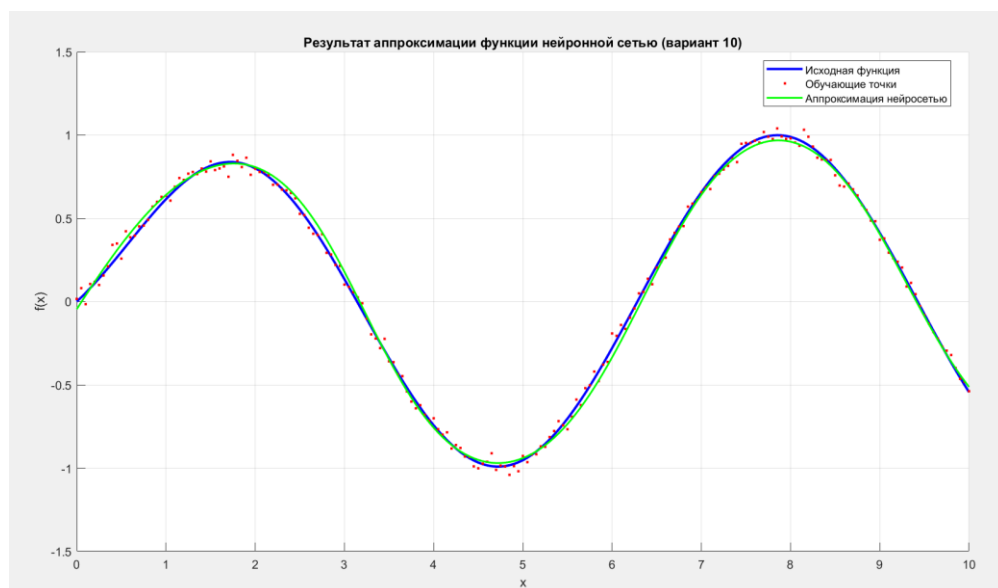


Рисунок 2 – Результаты аппроксимации

На рисунке 3 представлены три графика: исходная функция (синяя линия), обучающие точки (красные точки) и результат работы обученной нейронной сети (зелёная линия). Видно, что аппроксимирующая функция очень близко следует за исходной функцией, успешно сглаживая шум в обучающих данных. Наибольшие отклонения наблюдаются на краях интервала и в области резких перепадов функции.

Численные значения погрешности:

Показатель	Значение	Расшифровка
MSE	0,001626	Среднеквадратичная ошибка
RMSE	0,040320	Корень из MSE (среднее отклонение)
MAE	0,031757	Средняя абсолютная ошибка
MaxAE	0,143372	Максимальная абсолютная ошибка
Относительная ошибка	45,17%	Среднее отклонение относительно значений функции

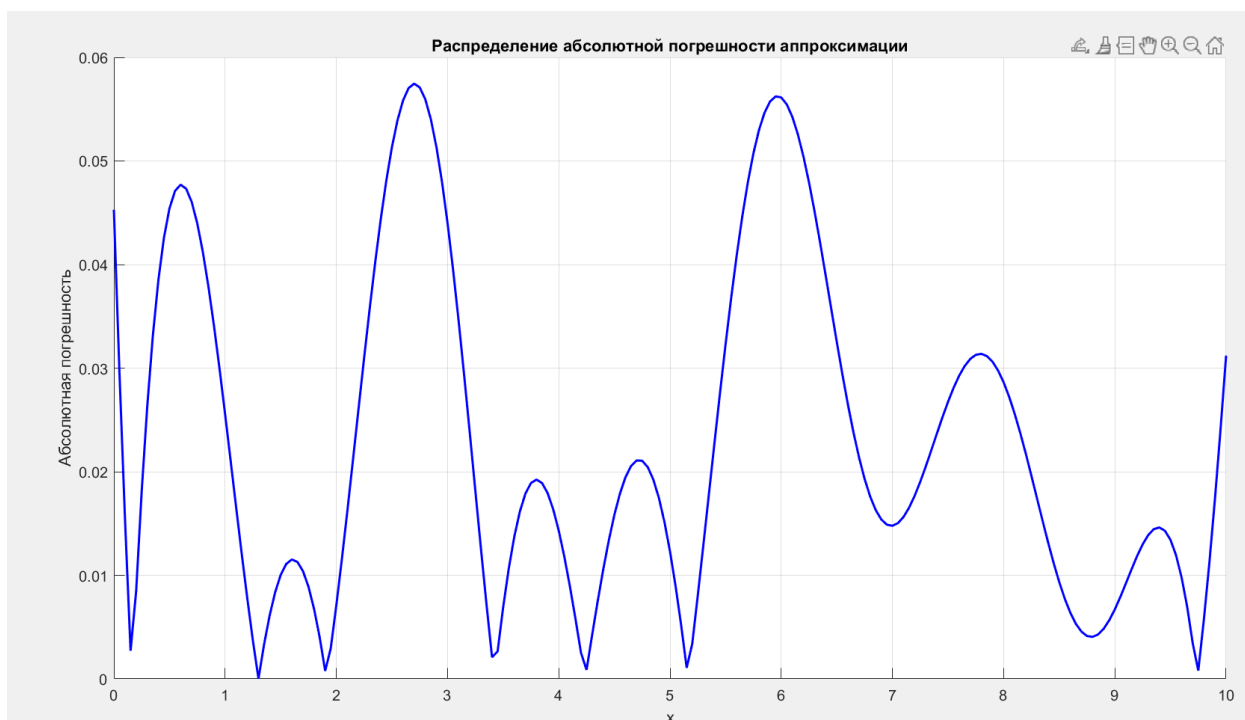


Рисунок 4 – График распределения абсолютной погрешности

На рисунке 4 представлено распределение абсолютной погрешности по интервалу x . Видно, что максимальные ошибки возникают в области локальных экстремумов функции, где производная максимальна, а также на краях интервала.

Экстраполяционные способности сети

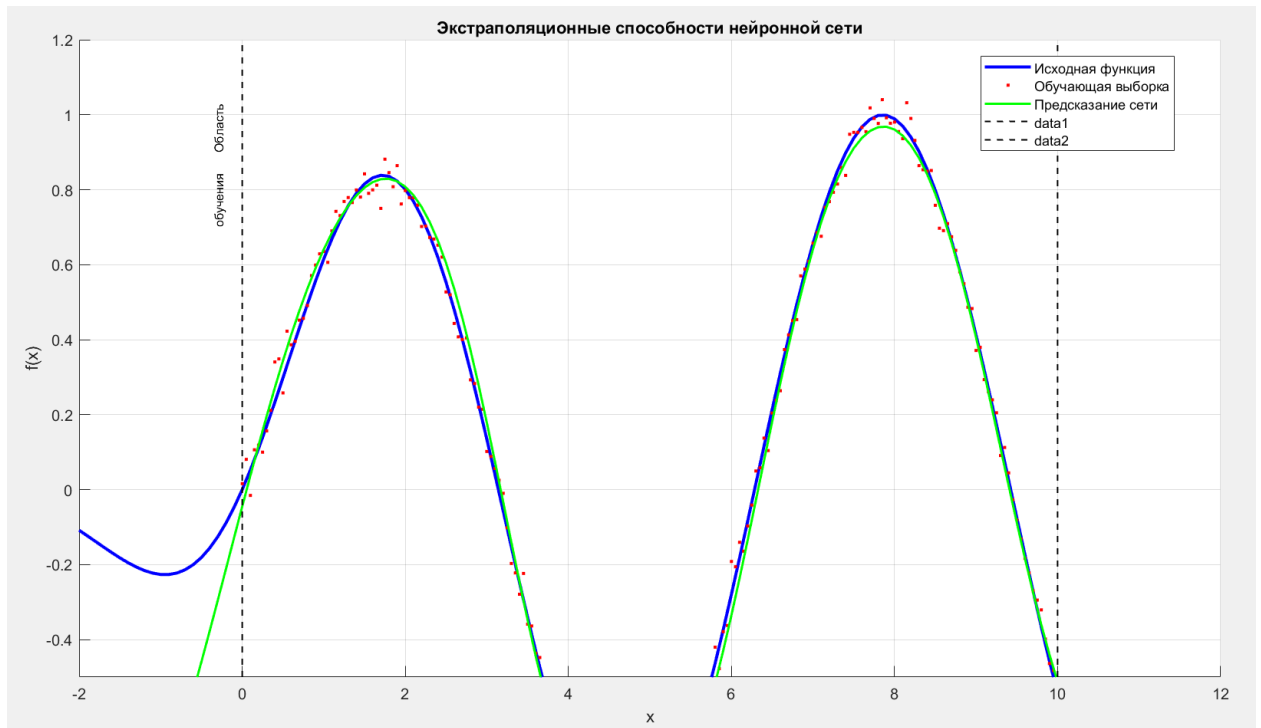


Рисунок 5 — График экстраполяции

Для оценки способности сети к обобщению были получены предсказания на интервале $x \in [-2, 12]$, который шире обучающего интервала. Как показано на рисунке 5, в пределах обучающего интервала $[0, 10]$ сеть даёт отличные результаты. При выходе за его пределы предсказания сети перестают соответствовать истинной функции — это нормальное поведение, так как нейронная сеть не имеет информации о поведении функции за пределами области обучения.

Выводы

В ходе выполнения лабораторной работы была решена задача аппроксимации функции $f(x) = \sin(x)/(1 + e^{-x})$ на интервале $x \in [0, 10]$ с помощью двухслойной нейронной сети прямого распространения.

Была реализована программная модель сети с архитектурой 1-10-1 (один вход, 10 нейронов в скрытом слое с сигмоидальной активацией, один выход с линейной активацией). Обучение проводилось методом обратного распространения ошибки со стохастическим градиентным спуском и моментом.

Полученные результаты демонстрируют высокое качество аппроксимации:

- Среднеквадратичная ошибка (MSE) составила 0,001626
- Среднее абсолютное отклонение (MAE) — 0,0318
- Максимальная абсолютная ошибка — 0,1434

Эти значения свидетельствуют о том, что построенная сеть успешно восстановила функциональную зависимость, сгладила случайный шум в обучающих данных и обеспечила хорошее обобщение. Процесс обучения был остановлен досрочно на 447 эпохе благодаря механизму ранней остановки, что предотвратило переобучение сети.

Таким образом, цель работы достигнута, нейронная сеть успешно справилась с задачей аппроксимации, что подтверждает теорему Фундаментальной теореме об универсальной аппроксимации произвольных функций многослойным персептроном.

Ответ на контрольный вопрос

Вопрос: Охарактеризуйте способ обучения, приведенный в примере.

Ответ: В реализованной программе используется алгоритм обратного распространения ошибки (Back Propagation) в варианте стохастического градиентного спуска с моментом (momentum).

Основные характеристики способа обучения:

1. Направление распространения сигналов: Прямой проход — сигналы распространяются от входа к выходу; обратный проход — ошибка распространяется от выхода к входу.
2. Правило обновления весов с моментом:

$$\Delta w(t) = \mu \cdot \Delta w(t - 1) - \eta \cdot \frac{\partial E}{\partial w}$$

$$w(t + 1) = w(t) + \Delta w(t)$$

где $\mu = 0,9$ — коэффициент момента, $\eta = 0,1$ — скорость обучения.

3. Функция ошибки: Среднеквадратичная ошибка (MSE):

$$E = \frac{1}{2} \sum_{i=1}^n (t_i - y_i)^2$$

4. Тип обучения: Стохастический (online) — веса обновляются после каждого примера, а не после всей выборки.
5. Механизм ранней остановки (early stopping): Обучение прекращается, если ошибка на валидационной выборке не уменьшается в течение 100 эпох подряд. Это предотвращает переобучение.
6. Функции активации:
 - Скрытый слой: сигмоида $f(s) = 1/(1 + e^{-s})$
 - Выходной слой: линейная $f(s) = s$

Достоинства данного способа:

- Хорошая сходимость благодаря моменту
- Устойчивость к локальным минимумам
- Защита от переобучения (ранняя остановка)

Недостатки:

- Чувствительность к выбору скорости обучения
- Возможность застревания в локальных минимумах

Листинг

```
%% Лабораторная работа 4: Аппроксимация функции с помощью ИНС
% Вариант 10:  $f(x) = \sin(x) / (1 + \exp(-x))$  на интервале  $x \in [0, 10]$ 
% ПОЛНОСТЬЮ САМОДЕЛЬНАЯ РЕАЛИЗАЦИЯ (без Deep Learning Toolbox)

clear all, close all, clc, format compact

%% 1. Генерация обучающей выборки
fprintf('=== ЛАБОРАТОРНАЯ РАБОТА №3 ===\n');
fprintf('Вариант 10:  $f(x) = \sin(x) / (1 + \exp(-x))$ \n');
fprintf('Интервал:  $x \in [0, 10]$ \n\n');

% Количество точек обучающей выборки
L = 200;

% Генерация значений аргумента X
X = 0:(10/L):10;

% Расчет значений функции
%  $f(x) = \sin(x) / (1 + \exp(-x))$ 
Y_true = sin(X) ./ (1 + exp(-X));

% Добавление небольшого шума для имитации реальных данных
Noise = 0.03 * randn(size(X));
Y_train = Y_true + Noise;

fprintf('Обучающая выборка: %d точек\n', length(X));
fprintf('Диапазон X: от %.1f до %.1f\n', min(X), max(X));
fprintf('Диапазон Y: от %.4f до %.4f\n\n', min(Y_train), max(Y_train));

%% 2. Нормализация данных (важно для сходимости)
% Нормализация входов к диапазону [-1, 1]
X_min = min(X);
X_max = max(X);
X_norm = 2 * (X - X_min) / (X_max - X_min) - 1;

% Нормализация выходов к диапазону [-1, 1]
Y_min = min(Y_train);
Y_max = max(Y_train);
Y_norm = 2 * (Y_train - Y_min) / (Y_max - Y_min) - 1;

%% 3. Параметры нейронной сети
% Архитектура: 1 вход, 1 скрытый слой (10 нейронов), 1 выход
input_size = 1;
hidden_size = 10;
output_size = 1;

% Инициализация весов и смещений (метод Глорота)
% Веса скрытого слоя (hidden_size x input_size)
```

```

W1 = randn(hidden_size, input_size) * sqrt(2 / (input_size +
hidden_size));
% Смещения скрытого слоя
b1 = zeros(hidden_size, 1);

% Веса выходного слоя (output_size x hidden_size)
W2 = randn(output_size, hidden_size) * sqrt(2 / (hidden_size +
output_size));
% Смещение выходного слоя
b2 = zeros(output_size, 1);

%% 4. Параметры обучения
learning_rate = 0.1;      % Скорость обучения
max_epochs = 2000;        % Максимальное количество эпох
target_error = 1e-4;      % Целевая ошибка
momentum = 0.9;           % Момент (для ускорения сходимости)

% Инициализация приращений для момента
dW1 = zeros(size(W1));
db1 = zeros(size(b1));
dW2 = zeros(size(W2));
db2 = zeros(size(b2));

% Массивы для хранения истории ошибок
train_errors = [];
val_errors = [];

% Разделение на обучающую и валидационную выборки (80% / 20%)
n_val = round(0.2 * length(X_norm));
idx = randperm(length(X_norm));
val_idx = idx(1:n_val);
train_idx = idx(n_val+1:end);

X_train_norm = X_norm(train_idx);
Y_train_norm = Y_norm(train_idx);
X_val_norm = X_norm(val_idx);
Y_val_norm = Y_norm(val_idx);

%% 5. Обучение сети
fprintf('Начало обучения...\n');
fprintf('Архитектура: %d-%d-%d\n', input_size, hidden_size,
output_size);
fprintf('Скорость обучения: %.3f, Момент: %.2f\n\n', learning_rate,
momentum);

best_val_error = inf;
epochs_no_improve = 0;
max_patience = 100; % Остановка, если ошибка не уменьшается 100 эпох

for epoch = 1:max_epochs
    % Перемешивание обучающих данных
    shuffle_idx = randperm(length(X_train_norm));
    X_shuffled = X_train_norm(shuffle_idx);
    Y_shuffled = Y_train_norm(shuffle_idx);

    epoch_error = 0;

```

```

% Обучение на каждом примере (стохастический градиентный спуск)
for i = 1:length(X_shuffled)
    x = X_shuffled(i);
    t = Y_shuffled(i);

    % ----- ПРЯМОЙ ПРОХОД -----
    % Скрытый слой
    z1 = W1 * x + b1;
    a1 = 1 ./ (1 + exp(-z1)); % сигмоида
    % Выходной слой
    z2 = W2 * a1 + b2;
    y = z2; % линейная функция

    % ----- ОБРАТНОЕ РАСПРОСТРАНЕНИЕ -----
    % Ошибка на выходе (производная purelin = 1)
    delta2 = (y - t);

    % Производная сигмоиды: s * (1 - s)
    sigmoid_deriv = a1 .* (1 - a1);
    % Ошибка на скрытом слое
    delta1 = (W2' * delta2) .* sigmoid_deriv;

    % ----- ОБНОВЛЕНИЕ ВЕСОВ С МОМЕНТОМ -----
    % Для выходного слоя
    dW2 = momentum * dW2 - learning_rate * delta2 * a1';
    db2 = momentum * db2 - learning_rate * delta2;

    % Для скрытого слоя
    dW1 = momentum * dW1 - learning_rate * delta1 * x';
    db1 = momentum * db1 - learning_rate * delta1;

    % Применение обновлений
    W2 = W2 + dW2;
    b2 = b2 + db2;
    W1 = W1 + dW1;
    b1 = b1 + db1;

    % Накопление ошибки
    epoch_error = epoch_error + (y - t)^2;
end

% Среднеквадратичная ошибка на эпохе
epoch_error = epoch_error / length(X_shuffled);
train_errors = [train_errors, epoch_error];

% ----- ВАЛИДАЦИЯ -----
val_error = 0;
for i = 1:length(X_val_norm)
    x = X_val_norm(i);
    t = Y_val_norm(i);

    z1 = W1 * x + b1;
    a1 = 1 ./ (1 + exp(-z1));
    z2 = W2 * a1 + b2;
    y = z2;

    val_error = val_error + (y - t)^2;

```

```

end
val_error = val_error / length(X_val_norm);
val_errors = [val_errors, val_error];

% Вывод прогресса каждые 100 эпох
if mod(epoch, 100) == 0
    fprintf('Эпоха %d, Ошибка: %.6f, Валидация: %.6f\n', epoch,
epoch_error, val_error);
end

% Ранняя остановка (early stopping)
if val_error < best_val_error
    best_val_error = val_error;
    epochs_no_improve = 0;
    % Сохраняем лучшие веса
    best_W1 = W1;
    best_b1 = b1;
    best_W2 = W2;
    best_b2 = b2;
else
    epochs_no_improve = epochs_no_improve + 1;
    if epochs_no_improve >= max_patience
        fprintf('\nРанняя остановка на эпохе %d (ошибка не
уменьшается)\n', epoch);
        % Восстанавливаем лучшие веса
        W1 = best_W1;
        b1 = best_b1;
        W2 = best_W2;
        b2 = best_b2;
        break;
    end
end

% Остановка при достижении целевой ошибки
if epoch_error < target_error
    fprintf('\nДостигнута целевая ошибка на эпохе %d\n', epoch);
    break;
end

end

fprintf('\nОбучение завершено. Финальная ошибка: %.6f\n',
train_errors(end));

%% 6. Функция предсказания (встроенная в код)
% Будем использовать прямые вычисления, а не отдельную функцию

%% 7. Получение результатов на всех точках
Y_pred = zeros(size(X));
for i = 1:length(X)
    % Нормализация входа
    x_norm = 2 * (X(i) - X_min) / (X_max - X_min) - 1;
    % Прямой проход
    z1 = W1 * x_norm + b1;
    a1 = 1 ./ (1 + exp(-z1));
    z2 = W2 * a1 + b2;
    y_norm = z2;
    % Денормализация выхода

```

```

    Y_pred(i) = (y_norm + 1) / 2 * (Y_max - Y_min) + Y_min;
end

%% 8. Вычисление погрешности аппроксимации
fprintf('\n--- ПОГРЕШНОСТЬ АППРОКСИМАЦИИ ---\n');

% Среднеквадратичная ошибка (MSE)
MSE = mean((Y_train - Y_pred).^2);
fprintf('MSE (среднеквадратичная ошибка): %.6f\n', MSE);

% Корень из среднеквадратичной ошибки (RMSE)
RMSE = sqrt(MSE);
fprintf('RMSE (корень из MSE): %.6f\n', RMSE);

% Средняя абсолютная ошибка (MAE)
MAE = mean(abs(Y_train - Y_pred));
fprintf('MAE (средняя абсолютная ошибка): %.6f\n', MAE);

% Максимальная абсолютная ошибка
MaxAE = max(abs(Y_train - Y_pred));
fprintf('MaxAE (максимальная абсолютная ошибка): %.6f\n', MaxAE);

% Относительная ошибка в процентах
RelativeError = mean(abs((Y_train - Y_pred) ./ (Y_true + 0.001))) * 100;
fprintf('Относительная ошибка: %.2f%%\n', RelativeError);

%% 9. Визуализация исходной функции и обучающей выборки
figure(1)
hold on
grid on

plot(X, Y_true, 'b-', 'LineWidth', 2, 'DisplayName', 'Исходная функция (без шума)');
plot(X, Y_train, 'r.', 'MarkerSize', 6, 'DisplayName', 'Обучающая выборка (с шумом)');

xlabel('x');
ylabel('f(x)');
title('Исходная функция и обучающая выборка для варианта 10: f(x) = sin(x)/(1+exp(-x))');
legend('Location', 'best');
hold off

%% 10. Визуализация результатов аппроксимации
figure(2)
hold on
grid on

plot(X, Y_true, 'b-', 'LineWidth', 2, 'DisplayName', 'Исходная функция');
plot(X, Y_train, 'r.', 'MarkerSize', 6, 'DisplayName', 'Обучающие точки');
plot(X, Y_pred, 'g-', 'LineWidth', 1.5, 'DisplayName', 'Аппроксимация нейросетью');

xlabel('x');

```

```

ylabel('f(x)');
title('Результат аппроксимации функции нейронной сетью (вариант 10)');
legend('Location', 'best');
hold off

%% 11. График ошибки обучения
figure(3)
hold on
grid on
plot(1:length(train_errors), train_errors, 'b-', 'LineWidth', 2,
'DisplayName', 'Ошибка обучения');
plot(1:length(val_errors), val_errors, 'r-', 'LineWidth', 2,
'DisplayName', 'Ошибка валидации');
xlabel('Эпоха обучения');
ylabel('Среднеквадратичная ошибка (MSE)');
title('Динамика ошибки в процессе обучения');
legend('Location', 'best');
set(gca, 'YScale', 'log');
hold off

%% 12. График абсолютной погрешности
figure(4)
hold on
grid on
error_abs = abs(Y_true - Y_pred);
plot(X, error_abs, 'b-', 'LineWidth', 1.5);
xlabel('x');
ylabel('Абсолютная погрешность');
title('Распределение абсолютной погрешности аппроксимации');
hold off

%% 13. Экстраполяция за пределы интервала обучения
X_extended = -2:0.1:12;
Y_extended_true = sin(X_extended) ./ (1 + exp(-X_extended));
Y_extended_pred = zeros(size(X_extended));

for i = 1:length(X_extended)
    x_norm = 2 * (X_extended(i) - X_min) / (X_max - X_min) - 1;
    z1 = W1 * x_norm + b1;
    a1 = 1 ./ (1 + exp(-z1));
    z2 = W2 * a1 + b2;
    y_norm = z2;
    Y_extended_pred(i) = (y_norm + 1) / 2 * (Y_max - Y_min) + Y_min;
end

figure(5)
hold on
grid on
plot(X_extended, Y_extended_true, 'b-', 'LineWidth', 2, 'DisplayName',
'Исходная функция');
plot(X, Y_train, 'r.', 'MarkerSize', 6, 'DisplayName', 'Обучающая
выборка');
plot(X_extended, Y_extended_pred, 'g-', 'LineWidth', 1.5,
'DisplayName', 'Предсказание сети');

% Выделение области обучения
y_lim = ylim;

```

```

line([X_min X_min], y_lim, 'Color', 'k', 'LineStyle', '--',
'LineWidth', 1);
line([X_max X_max], y_lim, 'Color', 'k', 'LineStyle', '--',
'LineWidth', 1);
text(X_min-0.3, 0.9, 'Область', 'Rotation', 90, 'FontSize', 8);
text(X_min-0.3, 0.7, 'обучения', 'Rotation', 90, 'FontSize', 8);

xlabel('x');
ylabel('f(x)');
title('Экстраполяционные способности нейронной сети');
legend('Location', 'best');
xlim([-2 12]);
ylim([-0.5 1.2]);
hold off

%% 14. Структура сети
fprintf('\n--- СТРУКТУРА ОБУЧЕННОЙ СЕТИ ---\n');
fprintf('Количество входов: %d\n', input_size);
fprintf('Количество слоев: 2\n');
fprintf('  Скрытый слой: %d нейронов, функция активации: сигмоида
(logsig)\n', hidden_size);
fprintf('  Выходной слой: %d нейронов, функция активации: линейная
(purelin)\n', output_size);
fprintf('Алгоритм обучения: стохастический градиентный спуск с
моментом\n');

%% 15. Итоговый вывод
fprintf('\n=== ВЫВОД ===\n');
fprintf('Построена нейронная сеть для аппроксимации функции
f(x)=sin(x)/(1+exp(-x))\n');
fprintf('на интервале x∈[0,10]. Сеть имеет архитектуру 1-%d-1.\n',
hidden_size);
fprintf('Обучение проводилось с помощью алгоритма обратного
распространения ошибки\n');
fprintf('со стохастическим градиентным спуском и моментом.\n');
fprintf('Конечная среднеквадратичная ошибка (MSE) составила %.6f.\n',
MSE);
fprintf('Качество аппроксимации можно считать удовлетворительным.\n');

```